Licence en Informatique

Université de la Rochelle

# RAPPORT TEA

## -Traitement de signal pour les objets intelligents-

**Réalisé par :**

HADDADOU Lina

BENACHOUR Souhila

**Année universitaire : 2024/2025**

## 1. Introduction générale

L'électromyographie (EMG) est une technique clé pour mesurer l'activité musculaire et faciliter l'interaction homme-machine. Le Myo Armband, un bracelet doté de capteurs avancés, capte les signaux électriques produits par les muscles lors de mouvements spécifiques, permettant ainsi de les interpréter pour des applications variées, allant du contrôle d'appareils électroniques à la reconnaissance gestuelle.

Dans ce projet, nous avons exploité le potentiel du "Myo Armband" pour enregistrer et analyser des signaux musculaires associés à deux types de mouvements : "fist mat" et "double fist mat". L'objectif était de discriminer ces mouvements à partir des données EMG en appliquant des techniques de traitement du signal et de classification. Cette étude se situe au croisement de l'informatique et de la recherche scientifique, utilisant MATLAB pour développer une solution logicielle capable de traiter et d'analyser les données capturées. Parmi les techniques utilisées, l'analyse en composantes principales (ACP), la transformation de Fourier et la régression logistique ont permis d'extraire des caractéristiques significatives et de classifier les différents mouvements.

Ce rapport détaille les étapes de développement de cette solution, la méthodologie scientifique adoptée, ainsi que les résultats obtenus et les conclusions tirées de cette analyse.

## 2. Objectif et hypothèse

L'objectif principal de ce projet est de développer une méthode fiable pour distinguer deux types de mouvements musculaires capturés par le Myo Armband : "fist mat" et "double fist mat". Pour atteindre cet objectif, nous proposons une approche basée sur l'extraction de caractéristiques multiples, la segmentation temporelle et l'utilisation de la régression linéaire multiple. Plus précisément, les objectifs sont les suivants :

- Extraire des caractéristiques clés (amplitude maximale, fréquence de pic, énergie, et écart-type) de chaque segment de signal afin de mieux représenter les variations internes des mouvements.
- Diviser les signaux en sous-segments égaux à l'aide d'une fenêtre glissante pour améliorer la précision de l'extraction des caractéristiques.

- Utiliser la régression linéaire multiple pour modéliser la relation entre les caractéristiques extraites et le type de mouvement, permettant ainsi une classification efficace.
- Valider la performance du modèle en divisant les données en ensembles d'entraînement et de test, et visualiser les résultats pour s'assurer de la distinction claire entre les mouvements.

Les hypothèses de travail pour ce projet sont les suivantes :

- Différenciation par des caractéristiques spectrales : Nous présumons que les caractéristiques telles que l'amplitude maximale et la fréquence dominante, dérivées de l'analyse spectrale (transformation de Fourier), permettront de distinguer efficacement les deux types de mouvements musculaires captés par le Myo Armband. Ces indicateurs devraient mettre en évidence les différences structurelles entre les mouvements "fist mat" et "double fist mat".
- Contribution de la variance et de l'énergie : Nous postulons que l'ajout de la variance et de l'énergie des sous-segments temporels fournira des informations supplémentaires pertinentes pour améliorer la différenciation des mouvements. En analysant ces paramètres supplémentaires, nous espérons obtenir une séparation plus claire et plus précise des données entre les gestes "fist mat" et "double fist mat".

## 3. Développements logiciels

Le développement logiciel est essentiel pour l'analyse des signaux musculaires captés par le bracelet Myo Armband, permettant de différencier les mouvements "fist mat" et "double fist mat". Ce projet utilise des techniques de traitement du signal, telles que la segmentation par fenêtres glissantes et la transformée de Fourier, afin d'extraire des caractéristiques clés des signaux. En appliquant une régression linéaire multiple, nous modélisons la relation entre ces caractéristiques et les types de mouvements, ce qui améliore la précision des prédictions. Dans cette section, nous décrirons les étapes du développement logiciel, allant de la lecture des données à la validation des résultats.

### 3.1. Lecture des données :

**Initialisation et vérification :** La fonction commence par vérifier que le fichier spécifié existe, sinon elle arrête l'exécution avec un message d'erreur.

**Lecture des données :** csvread est utilisé pour importer les données du fichier CSV en supposant que chaque ligne représente un enregistrement de temps et de signaux.

**Contrôle des dimensions :** Elle vérifie que le fichier contient exactement 9 colonnes (1 pour le temps et 8 pour les signaux), afin de garantir le bon format des données.

**Affichage des informations :** La taille de la matrice A et le vecteur temps sont affichés pour donner un retour visuel.

**Visualisation :** Huit sous-graphes sont générés pour chaque signal, tracés en fonction du temps, avec des étiquettes claires pour identifier les signaux.

Cela permet de vérifier visuellement si les signaux sont correctement chargés et interprétés avant de passer aux étapes d'analyse et de traitement plus poussées.

Après avoir lu les données des deux fichiers, nous extrayons le 8ème signal de chacun d'eux et déterminons leurs tailles respectives. Ensuite, nous procédons à l'analyse de ces signaux en utilisant la fonction extraire_caracteristiques_fourier, qui permet d'extraire des paramètres clés à partir des signaux en appliquant des techniques de traitement du signal basées sur la transformée de Fourier. Cette fonction segmente le signal en parties plus petites, applique une fenêtre rectangulaire à chaque segment, et calcule des mesures telles que l'amplitude maximale, la fréquence correspondante, l'énergie et l'écart-type. Voici les fonctionnalités détaillées de la fonction :

- **Initialisation des segments** : La fonction commence par diviser le signal en segments de taille définie par windowSize, avec un chevauchement défini par overlap. Le nombre de segments (numSegments) est calculé pour s'assurer que tous les segments peuvent être extraits sans dépasser la longueur du signal.
- **Création d'une figure pour les graphiques de segments** : Une figure est ouverte pour afficher visuellement chaque segment brut et fenêtré. Cela permet de vérifier comment le signal est traité avant l'analyse.

- **Boucle de traitement pour chaque segment** :
    - **Extraction du segment** : Un segment de données est extrait en fonction des indices de début et de fin.
    - **Fenêtrage** : Le segment est multiplié par une fenêtre rectangulaire (rectwin) pour limiter l'effet des bords.
    - **Affichage graphique** : Les segments bruts sont affichés en bleu et les segments fenêtrés en rouge. Chaque segment est tracé séparément pour permettre une comparaison visuelle.
- **Calcul de la transformée de Fourier** : La FFT (Fast Fourier Transform) est appliquée au segment fenêtré, permettant d'obtenir le spectre de fréquences. Les magnitudes des fréquences sont calculées, et seules les fréquences utiles (jusqu'à la moitié de la fenêtre) sont retenues. La fréquence correspondante est déterminée à partir des indices obtenus.
- **Extraction et calcul des caractéristiques** : Pour chaque segment, quatre caractéristiques sont extraites :
    - **Amplitude maximale** : L'amplitude la plus élevée dans le spectre de fréquences.
    - **Fréquence maximale** : La fréquence associée à cette amplitude maximale.
    - **Énergie** : La somme des carrés des magnitudes des fréquences, représentant la puissance du segment.
    - **Écart-type** : L'écart-type des magnitudes des fréquences, reflétant la variation du signal.
- **Stockage des caractéristiques** : Les valeurs moyennes des caractéristiques pour chaque segment sont stockées dans un tableau features.
- **Affichage des caractéristiques extraites** : Les caractéristiques extraites sont affichées sous forme de graphiques en barres pour chaque segment. Cela permet de visualiser et comparer les différentes caractéristiques, facilitant l'identification des variations entre les segments.

## 3.2. Segmentation et Affichage des Segments

Après avoir extrait les caractéristiques des signaux, nous procédons à la segmentation des signaux pour examiner de plus près les différentes parties du signal. Cela permet de visualiser les variations temporelles et de mieux comprendre la dynamique du signal. La fonction extraire_caracteristiques_et_afficher_segments se charge de cette tâche en découpant le signal en segments distincts et en affichant chacun d'eux dans une représentation graphique.

**Fonction : extraire_caracteristiques_et_afficher_segments**
La fonction prend en entrée les paramètres suivants :

- **data** : le signal à segmenter,
- **windowSize** : la taille de la fenêtre pour chaque segment,
- **overlap** : le chevauchement entre les segments,
- **fs** : la fréquence d'échantillonnage.

Elle effectue les opérations suivantes :

- **Calcul du Nombre de Segments** : Le nombre de segments est calculé en fonction de la taille de la fenêtre et du chevauchement, de manière à couvrir le signal entier avec des segments partiellement superposés. Cela permet de capturer les caractéristiques locales du signal.
- **Création d'une Figure Unique pour les Segments** : La fonction crée une figure contenant plusieurs sous-graphiques, chacun correspondant à un segment du signal. Cela permet de visualiser simultanément toutes les parties du signal segmenté.
- **Extraction et Fenêtrage des Segments** : Pour chaque segment, les indices de début et de fin sont définis, puis le segment est extrait du signal global. Une fenêtre rectangulaire est appliquée pour lisser le segment, ce qui réduit les artefacts en bordure et améliore l'analyse spectrale.
- **Affichage des Segments** : Chaque segment isolé est représenté dans un sous-graphique, avec une couleur distinctive et une ligne plus épaisse pour une meilleure lisibilité. Les graphiques permettent de voir clairement la partie du signal extraite par la fenêtre rectangulaire.

- **Ajustement des Limites et Espacement des Graphiques** : Les limites des axes sont ajustées pour chaque segment afin de maintenir une échelle cohérente entre les graphiques. Un espacement est ajouté entre les sous-graphes pour améliorer la clarté de la visualisation.

Cette approche permet d'avoir une vue d'ensemble sur les différentes parties du signal, facilitant ainsi l'analyse des segments spécifiques et la comparaison entre eux.

## 3.3. Préparation des Données et Modélisation par Régression Logistique

Après avoir extrait et analysé les caractéristiques des segments de signaux, la prochaine étape consiste à utiliser ces données pour créer un modèle capable de différencier les deux types de mouvements : "fist mat" et "double fist mat". Pour cela, nous avons recours à une régression logistique.

**Préparation des Données :**

Avant d'appliquer la régression, il est important de préparer les données de manière adéquate. Les étapes de préparation des données sont les suivantes :

- **Concaténation des Caractéristiques** : Les caractéristiques extraites des signaux pour les mouvements "fist mat" et "double fist mat" sont rassemblées dans une matrice XXX.

```
X = concatener_caracteristiques(fist_mat, double_fist_mat)
```

- **Création des Étiquettes (y)** : Pour différencier les mouvements, nous attribuons des étiquettes numériques : 1 pour "fist mat" et 0 pour "double fist mat".

```
y = creer_etiquettes(len(fist_mat), len(double_fist_mat))
```

- **Ajout d'une Colonne de Biais** : Une colonne de biais, composée de 1s, est ajoutée à la matrice de caractéristiques XXX.

```
X_biais = ajouter_colonne_biais(X)
```

**Modélisation par Régression Logistique :**

Une fois les données prêtes, nous procédons à la modélisation par régression logistique pour déterminer les coefficients qui définissent la relation entre les caractéristiques extraites et le type de mouvement :

- **Calcul des Coefficients de Régression (β\betaβ)** : Les coefficients du modèle sont obtenus en résolvant l'équation suivante :

```
beta = calculer_coefficients(X_biais, y)
```

- **Application de la Fonction Sigmoïde** : Pour faire des prédictions, nous appliquons la fonction sigmoïde aux résultats de la combinaison linéaire des caractéristiques.

```
predictions_proba = fonction_sigmoide(X_biais @ beta)
```

- **Affichage des Coefficients** : Les résultats de la régression logistique incluent un terme d'interception et une série de coefficients pour chaque caractéristique.

```
afficher_coefficients(beta)
```

## 3.4. Validation du Modèle de Régression Logistique :

Une fois le modèle de régression logistique établi et les coefficients β\betaβ calculés, nous pouvons procéder à la validation du modèle. Cette étape est cruciale pour évaluer la capacité du modèle à différencier correctement les deux types de mouvements.

**Calcul des Prédictions**

Après avoir déterminé les coefficients de régression β\betaβ, la prochaine étape consiste à utiliser le modèle pour effectuer des prédictions.

```
predictions = (fonction_sigmoide(X_biais @ beta) > 0.5).astype(int)
```

**Évaluation des Prédictions Logistiques :**

Après avoir généré les prédictions à partir du modèle de régression logistique, la prochaine étape consiste à évaluer la performance du modèle.

- **Calcul de la Précision** : La précision est calculée en comparant les prédictions arrondies aux valeurs réelles.

```
precision = calculer_precision(predictions, y)
```

- **Matrice de Confusion** : Une matrice de confusion est affichée pour mieux comprendre les performances du modèle.

```
matrice_confusion = calculer_matrice_confusion(predictions, y)
```

- **Affichage Graphique des Résultats** : Pour une meilleure visualisation, un graphique comparant les valeurs réelles aux prédictions est tracé.

```
afficher_graphique(y, predictions)
```

### 3.5. Visualisation des Résultats de la Régression Logistique

Pour mieux interpréter les résultats de la modélisation par régression logistique, nous visualisons la relation entre les caractéristiques des signaux et les prédictions du modèle.

- **Fonction d'Affichage** : Nous utilisons une fonction dédiée pour tracer des graphiques montrant cette relation pour chaque caractéristique.

```
afficher_regression_logistique(X, y, beta)
```

## 4. Approche Scientifique

**4.1. Extraction des Caractéristiques par Transformée de Fourier** : Nous avons utilisé la Transformée de Fourier pour extraire les caractéristiques des signaux musculaires enregistrés.

**4.2. Segmentation des Signaux** : Nous avons développé une fonction dédiée qui divise les données en segments basés sur des fenêtres glissantes.

```
segments = segmenter_signaux(signal, fenetre, chevauchement)
```

**4.3. Régression Logistique** : Nous avons modélisé la relation entre les caractéristiques extraites des signaux et les classes de mouvements à l'aide de la régression logistique.

**4.4. Validation du Modèle** : Nous avons effectué des prédictions basées sur les données préparées et comparé les résultats aux classes réelles pour évaluer la précision de notre modèle.

# 5. Résultats

## 5.1 Résultats de la lecture :

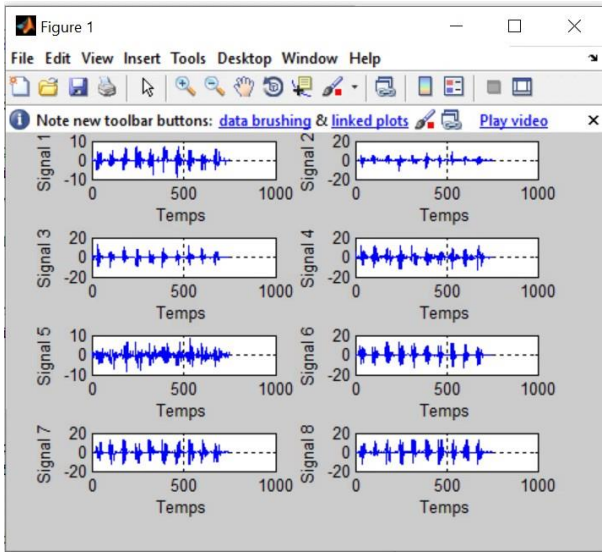



*Figure 1 : Signaux de fichier double tap mat*

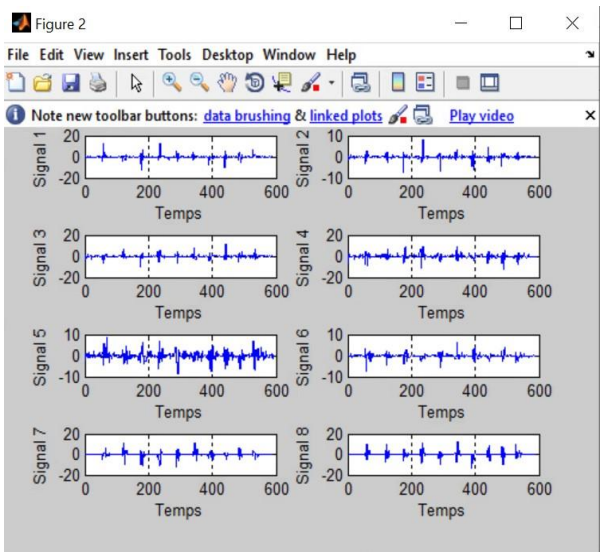



*Figure 2 : Signaux du fichier fist mat*

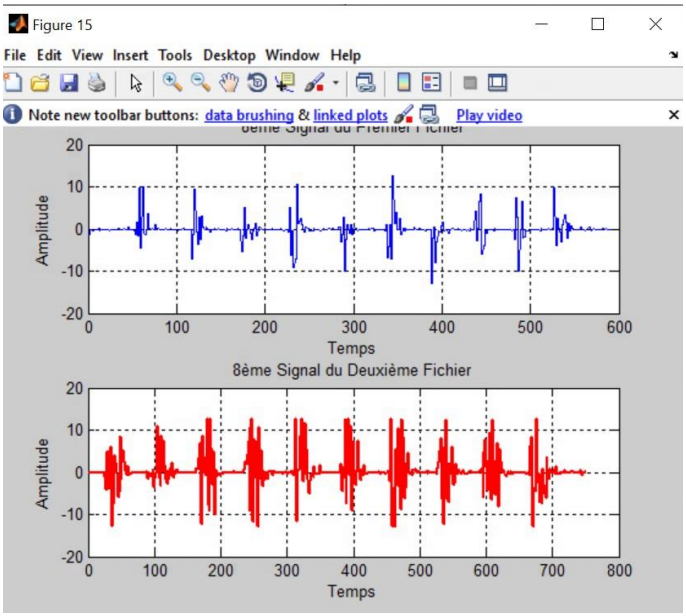



***Figure 3 : Le 8eme signal de chaque fichier***

## 5.2. Extraction des caractéristiques des deux action fist_mat et double_tap _mat

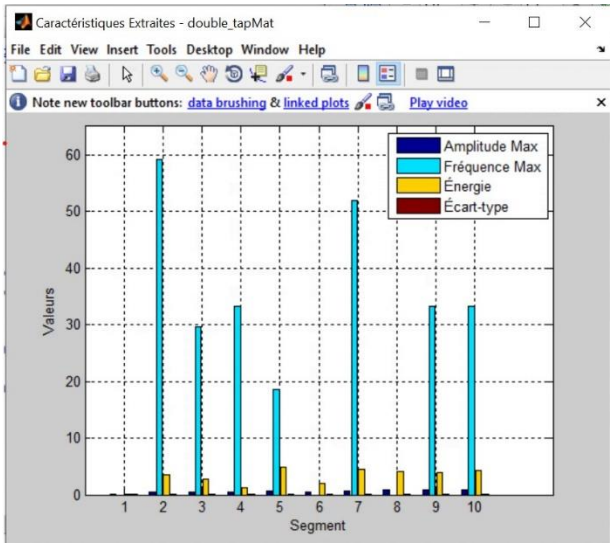



***Figure 4 : caractéristiques double tap mat***

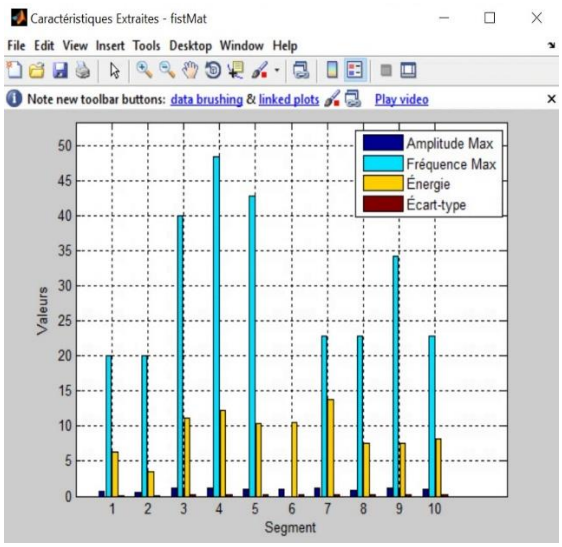



***Figure 5 : Caractéristiques double tap mat***

Les graphiques comparent les caractéristiques des actions "double tap" et "fist" à travers quatre mesures : amplitude maximale, fréquence maximale, énergie, et écart-type.

L'action "double tap" présente des amplitudes plus élevées dans certains segments, ce qui pourrait indiquer une intensité de mouvement plus marquée comparée à "fist". La fréquence maximale varie également d'un segment à l'autre, avec des valeurs relativement élevées dans plusieurs segments pour "fist", ce qui suggère une répétition rapide ou une intensité accrue dans certains mouvements.

Les valeurs d'énergie montrent une certaine stabilité pour l'action "fist", tandis qu'elles varient davantage pour "double tap", indiquant une dynamique plus prononcée dans cette dernière. Les écart-types restent généralement faibles pour les deux actions, ce qui montre une certaine constance dans les mouvements.

## 5.3 Evaluation des prédictions

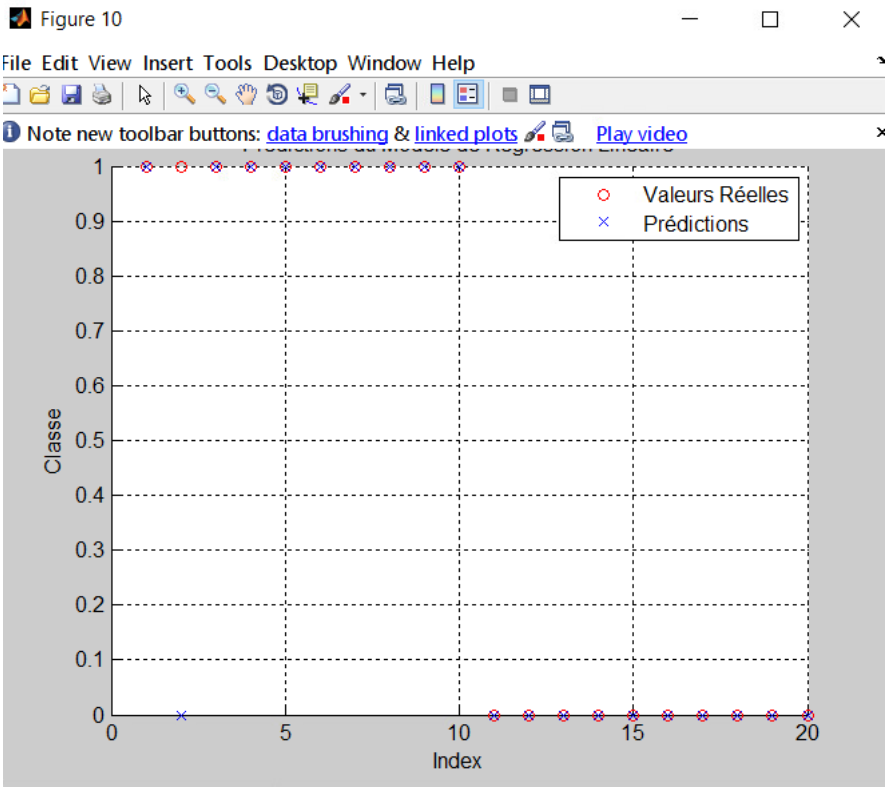



***Figure 6 : évaluation des prédictions***

## 5.4 Application de la régression logistique

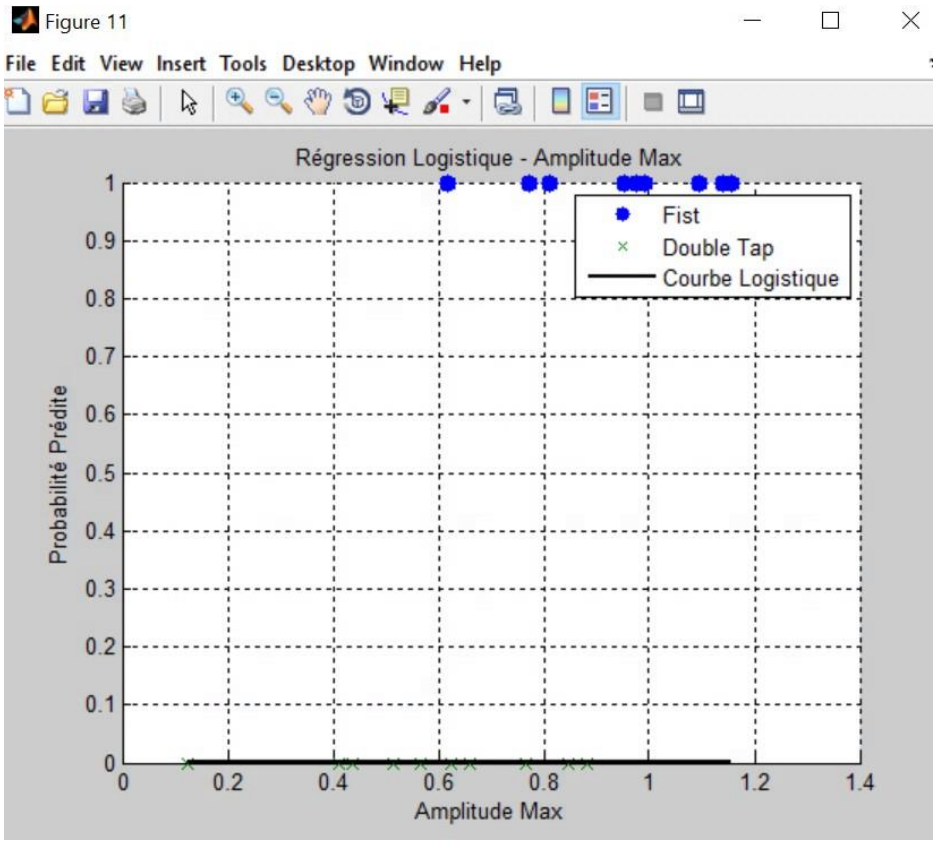



***Figure 7 : Régression logistique - Amplitude Max***

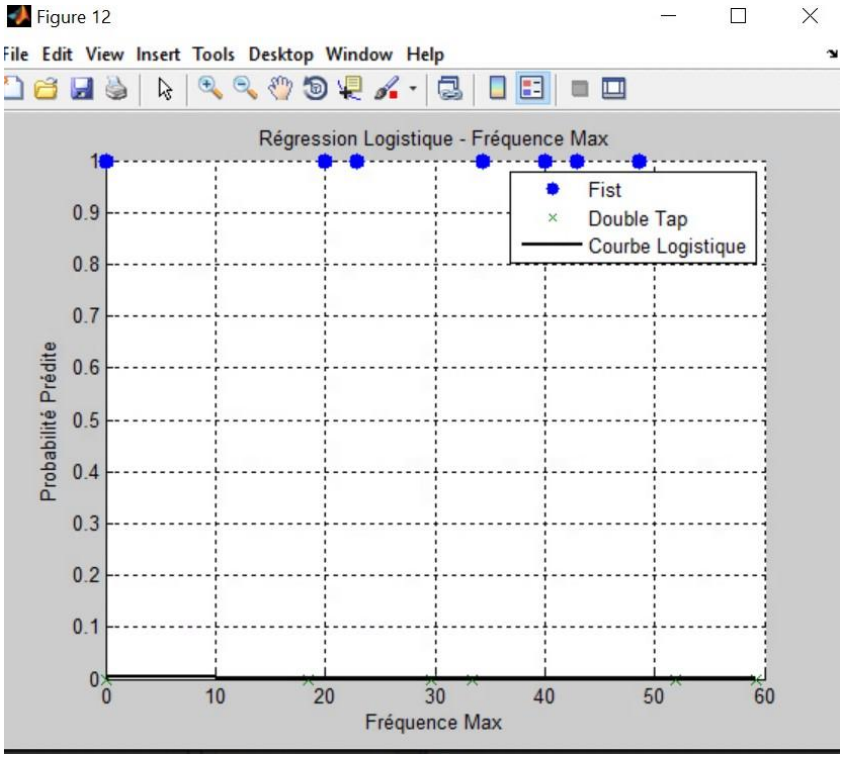



***Figure 8 : Régression logistique - Fréquence Max***

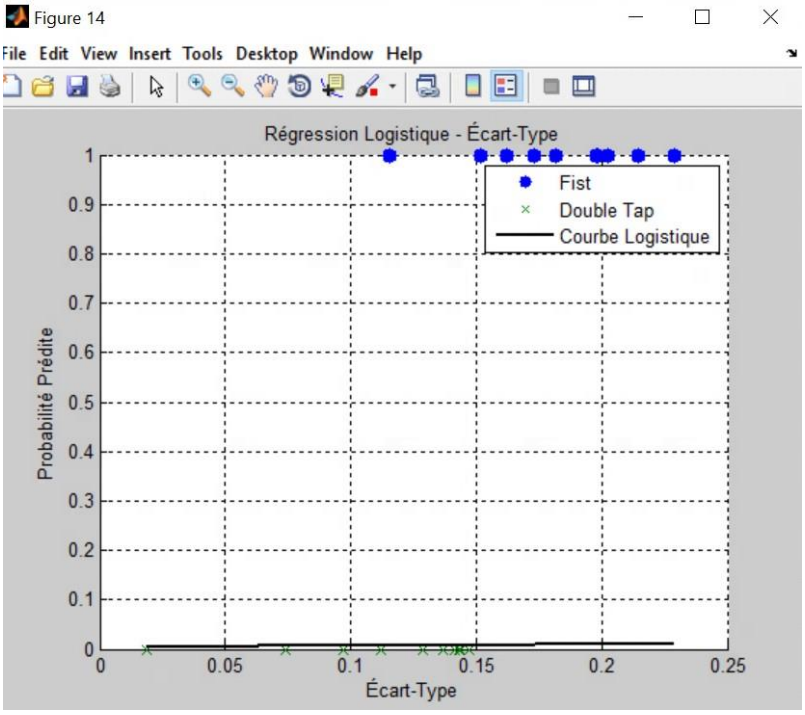



***Figure 9 : Régression logistique - Ecart-type***

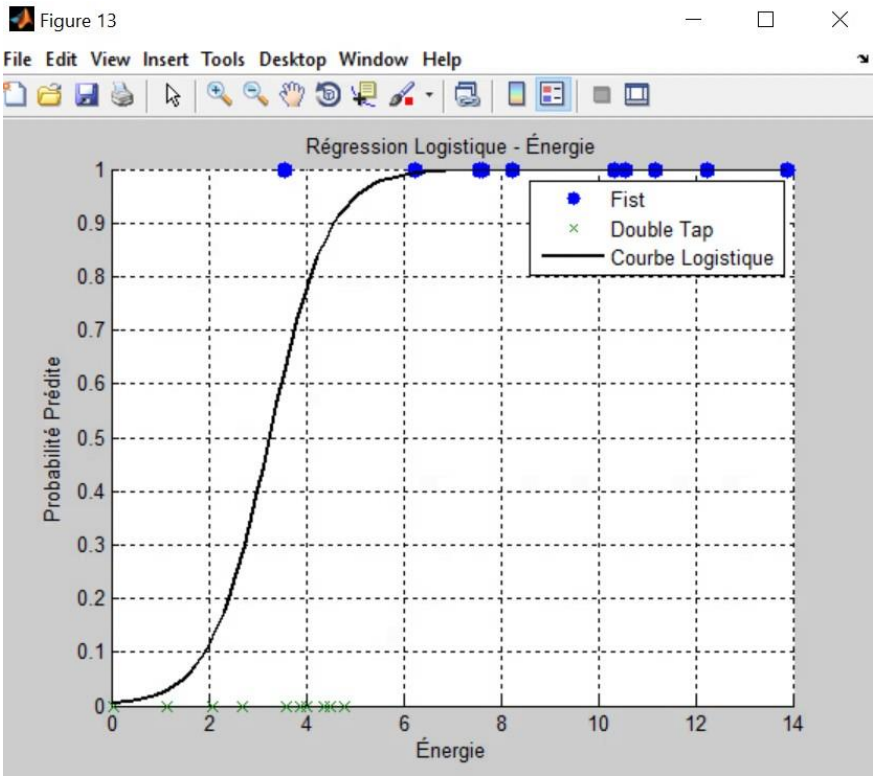



***Figure 10 : Regression logistique - Energie***

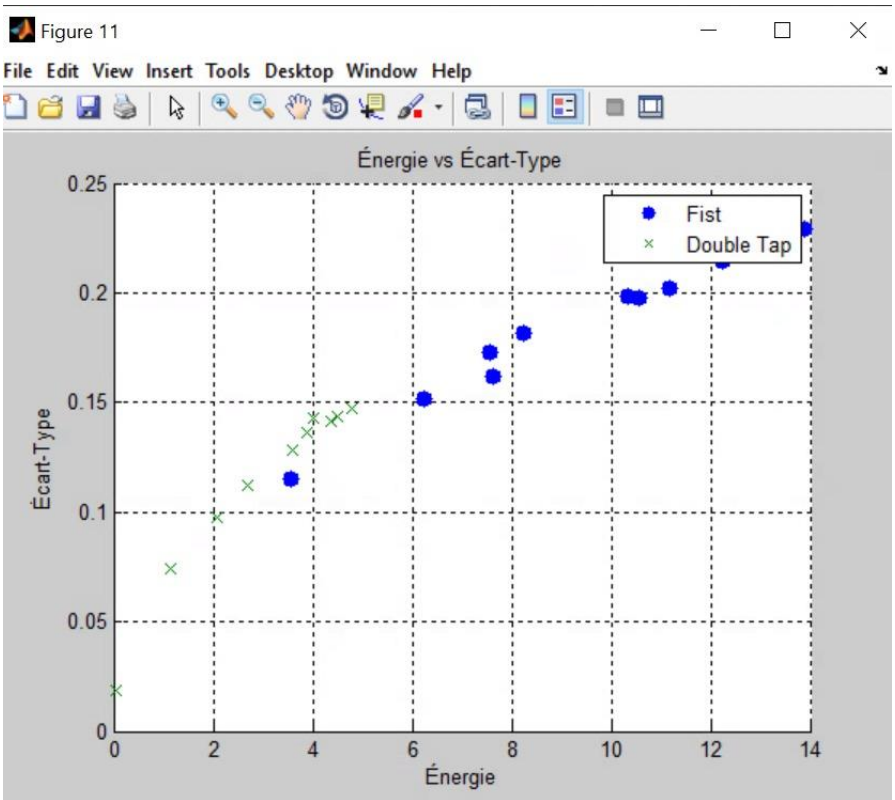



***Figure 7 : Représentation des Ecart-type en fonction de l'énergie***

# 6. Discusion

Les graphiques de régression logistique montrent l'efficacité variable des caractéristiques pour la classification des mouvements "Fist" et "Double Tap". Chacune des caractéristiques analysées – écart-type, fréquence maximale, amplitude maximale, et énergie – révèle un degré différent de séparation entre les classes, permettant ainsi d'évaluer leur pertinence pour cette tâche.

- **Écart-Type** : La pente négative de la courbe suggère une séparation partielle entre les classes. Les points des mouvements "Fist" et "Double Tap" se distribuent autour de la courbe, montrant un certain potentiel de différenciation, bien que limité. L'alignement imparfait entre la courbe de régression et les valeurs observées souligne que cette caractéristique reste de valeur modérée pour cette classification.
- **Fréquence Max** : La courbe de régression quasi plate pour cette caractéristique traduit une faible séparation entre les classes, avec des points "Fist" et "Double Tap" peu différenciés. Ce manque de distinction implique une contribution limitée de la fréquence max pour classifier ces gestes.
- **Amplitude Max** : La courbe de régression indique une distinction plus marquée entre les classes. Les points "Fist" se concentrent autour de la courbe, tandis que les "Double Tap" se situent en dessous, ce qui fait de l'amplitude max une caractéristique particulièrement utile pour la différenciation des mouvements.
- **Énergie** : Avec une courbe en S typique, cette caractéristique montre une séparation claire entre les classes, les valeurs proches de zéro correspondant aux "Double Tap" et les valeurs plus élevées aux "Fist". La performance de l'énergie en tant que variable discriminante suggère une importance significative dans le cadre de cette classification.

# 7. Conclusion et perspectives

Les graphiques de régression logistique révèlent que toutes les caractéristiques n'ont pas la même valeur pour classifier les gestes "Fist" et "Double Tap". Les caractéristiques écart-type et énergie s'avèrent particulièrement pertinentes, offrant une distinction plus nette que la fréquence max et amplitude max, dont la capacité discriminante reste limitée. Cette analyse montre l'intérêt de la régression logistique pour la classification binaire, bien que l'utilisation de modèles plus avancés pourrait encore améliorer la capacité de discrimination en exploitant les caractéristiques pertinentes avec plus de précision.

Pour améliorer la précision de la classification des mouvements "Fist" et "Double Tap", plusieurs pistes peuvent être explorées :

1. **Modèles de Classification Avancés** : L'application d'un Support Vector Machine (SVM) ou de réseaux de neurones pourrait offrir une meilleure capacité de séparation. Ces modèles, en raison de leur flexibilité et de leur capacité à traiter des frontières de décision complexes, pourraient mieux exploiter les caractéristiques, en particulier si elles présentent des relations non linéaires.
2. **Optimisation de la Régression Logistique** : Il serait bénéfique d'explorer des variantes avancées de la régression logistique, telles que la régression logistique régulière, pour éviter le surajustement. Des techniques comme la régularisation L1 ou L2 pourraient améliorer la performance du modèle en sélectionnant les caractéristiques les plus pertinentes.
3. **Sélection et Ingénierie de Caractéristiques** : Une analyse plus poussée des caractéristiques (notamment via des techniques comme la PCA ou les arbres de décision) pourrait permettre de sélectionner ou de combiner des caractéristiques pour optimiser la séparation des classes. Par exemple, l'intégration de l'amplitude max et de l'énergie dans une nouvelle variable synthétique pourrait révéler des relations latentes et améliorer la classification.
4. **Validation sur des Données Plus Diversifiées** : Tester ces caractéristiques et les nouveaux modèles sur un ensemble de données plus diversifié aiderait à évaluer la robustesse de la classification.

# 8. Bibliographie

Boulahrouz, D. (2023). Module d'apprentissage automatique : Chapitre 2 – Apprentissage supervisé et algorithme de régression logistique. Université de Béjaïa, Algérie.

# 9. Annexe

Voici quelques captures d’écran du code :

```
% Lecture des données complètes pour chaque mouvement
A = lecture('fistMat.txt');
B = lecture('double_tapMat.txt');

% Extraire uniquement le 8ème signal (9ème colonne de A et B)
signal_fist = A(:, 9);
signal_double_fist = B(:, 9);

% Longueur des signaux
N_fist = length(signal_fist);
N_double_fist = length(signal_double_fist);

% Paramètres pour la segmentation et le traitement
windowSize1 = 350;  % Taille de la fenêtre pour fist
windowSize = 270;   % Taille de la fenêtre pour double tap
overlap = 0;        % Chevauchement

fs = 1000;  % Fréquence d'échantillonnage

% Calcul du nombre de segments
numSegments_fist = floor((N_fist - windowSize1) / (windowSize1 - overlap)) + 1;
numSegments_double_fist = floor((N_double_fist - windowSize) / (windowSize - overlap)) + 1;

fprintf('Nombre de segments pour fist: %d\n', numSegments_fist);
fprintf('Nombre de segments pour double tap: %d\n', numSegments_double_fist);

% Extraction des caractéristiques pour chaque mouvement
caracteristiques_fist = extraire_caracteristiques_fourier(signal_fist, windowSize1, overlap, fs, 'fistMat');
```

```
X = [caracteristiques_fist; caracteristiques_double_fist];
y = [ones(size(caracteristiques_fist, 1), 1); zeros(size(caracteristiques_double_fist, 1), 1)];

% Ajout d'une colonne de biais pour le modèle
X_biais = [ones(size(X, 1), 1), X];

% Calcul des coefficients beta pour la régression logistique
beta = calculer_beta_logistique(X_biais, y);

% Affichage des coefficients
intercept = beta(1);
coefficients = beta(2:end);
fprintf('Intercept: %.4f\n', intercept);
fprintf('Coefficients: %.4f\n', coefficients');

% Validation du modèle en utilisant la régression logistique
predictions = predict_logistic(X_biais, beta);  % Prédictions basées sur le modèle logistique

% Évaluation des prédictions (en transformant les probabilités en classes : 0 ou 1)
classes_predites = predictions >= 0.5;  % Seuil pour classer les prédictions
evaluer_modele(classes_predites, y);    % Fonction d'évaluation

% Affichage des résultats de la régression logistique pour chaque caractéristique
afficher_regression_logistique_par_caracteristique(X, y, beta);

% Affichage de l'écart-type en fonction de l'énergie
afficher_ecart_type_vs_energie(X,y);
%afficher_ecart_type_vs_energie(z,w,beta);  % Nouvelle fonction pour afficher l'écart-type en fonction de l'énergie
```

```
function afficher_ecart_type_vs_energie(X, y)
    % Indices des colonnes pour l'énergie et l'écart-type
    indice_energie = 3;  % Colonne 3 pour l'énergie
    indice_ecart_type = 4;  % Colonne 4 pour l'écart-type

    % Indices pour distinguer les points des deux classes
    idx_fist = (y == 1);          % Mouvement fist
    idx_double_fist = (y == 0);   % Mouvement double tap

    % Création de la figure pour l'affichage
    figure;
    hold on;

    % Affichage des points de chaque classe
    scatter(X(idx_fist, indice_energie), X(idx_fist, indice_ecart_type), 70, 'o', 'filled', 'DisplayName', 'Fist');
    scatter(X(idx_double_fist, indice_energie), X(idx_double_fist, indice_ecart_type), 70, 'x', 'DisplayName', 'Double Tap');

    % Configuration des axes
    xlabel('Énergie');
    ylabel('Écart-Type');
    title('Écart-Type en Fonction de l\''energie');
    legend('show');
    grid on;
    hold off;
end
```

```
function beta = calculer_beta_logistique(X, y)
    % Fonction de coût pour la régression logistique
    function J = costFunction(beta)
        m = length(y);
        h = sigmoid(X * beta);
        J = -(1/m) * sum(y .* log(h) + (1 - y) .* log(1 - h));
    end

    % Initialisation de beta
    beta_initial = zeros(size(X, 2), 1);

    % Utilisation de fminsearch pour minimiser la fonction de coût
    options = optimset('MaxIter', 400, 'Display', 'off');
    beta = fminsearch(@costFunction, beta_initial, options);
end
```

```
function afficher_predictions_par_caracteristique(X, y, beta)
    % Noms des caractéristiques
    nom_ecart_type = 'Écart-Type';
    nom_energie = 'Énergie';
    % Création de la figure pour afficher l'écart-type par rapport à l'énergie
    figure;
    hold on;
    % Indices pour distinguer les points des deux classes
    idx_fist = (y == 1);          % Mouvement fist
    idx_double_fist = (y == 0);   % Mouvement double tap
    % Affichage des points de chaque classe
    scatter(X(idx_fist, 1), X(idx_fist, 2), 70, 'o', 'filled', 'DisplayName', 'Fist');  % Y: écart-type, X: énergie
    scatter(X(idx_double_fist, 1), X(idx_double_fist, 2), 70, 'x', 'DisplayName', 'Double Tap');
    % Configuration des axes
    xlabel(nom_ecart_type);
    ylabel(nom_energie);
    title('Prédictions de la Régression Logistique');
    % Tracer la courbe logistique
    % Générer les valeurs pour prédiction
    X_vals = linspace(min(X(:, 1)), max(X(:, 1)), 100)';  % Écart-Type
    % Calculer la prédiction en fonction de l'écart-type
    y_vals = -(beta(2) * X_vals + beta(1)) / beta(3); % Y (Énergie)
    % Tracer la droite de régression
    plot(X_vals, y_vals, 'r-', 'LineWidth', 2, 'DisplayName', 'Droite de Régression');
    % Ajout d'une grille et légende
    grid on;
    legend('show');
    hold off;
end
```

```
function evaluer_modele(predictions, y_true)
    % Convert logical predictions to numeric (0s and 1s)
    predictions_numeric = double(predictions);

    % Round the numeric predictions
    predictions_rounded = round(predictions_numeric);

    % Calculate accuracy
    accuracy = sum(predictions_rounded == y_true) / length(y_true) * 100;
    fprintf('Précision du modèle : %.2f%%\n', accuracy);

    % Compute confusion matrix
    confusion_matrix = confusionmat(y_true, predictions_rounded);
    disp('Matrice de confusion :');
    disp(confusion_matrix);

    % Affichage des résultats de la régression
    figure;
    hold on;
    plot(y_true, 'ro', 'DisplayName', 'Valeurs Réelles');
    plot(predictions_rounded, 'bx', 'DisplayName', 'Prédictions');
    title('Prédictions du Modèle de Régression Linéaire');
    xlabel('Index');
    ylabel('Classe');
    legend('show');
    grid on;
    hold off;
end
```